集团 Token分析及精细化管控

背景

随着大模型、Agent、Harness 等能力的提升，大部分企业都开始将这些AI 大模型落地到企业日常办公和日常流程中。但是，Opus 等定期模型的费用也居高不下，Harness 等消耗的Token也越来越多，我们企业 6 月 1 日（3W 美金的消费）、6 月 2 日（4W 美金的消费）。

大部分企业，开始精细化的管控 Token 的使用，例如：

微软取消了 Claude Code 和相关模型的使用：微软按下 vibe coding 暂停键：烧 token 已经比员工贵了

米哈游Token 使用：米哈游一夜烧掉200万元Token，大厂高管也开始质疑：Token烧不出价值，但养肥了谁?

问题

如何实现：既能使用好的模型（GPT 系列、Claude 系列）有能够有效的控制 Token 的消耗量?

内部地址：https://gitlab.gz.cvte.cn/wa-ai/user_token
观察地址：https://langfuse.cvte.com/project/cmpmbq8jm00ynz40708943s7a/dashboards

例如 Agent 的优化：

- op-my-pi 的方式
- agent 优化：小龙虾彻底凉了？清华团队连夜开源Agent神器，Token成本狂降70%

例如提示词优化&上下文优化：

- 腾讯开源Agent Memory，让Token消耗降低61%
- 让 Claude Code 成本爆降 80%，这个开源工具有点猛...
- 告别 Token 刺客！开源上下文优化层 Headroom 如何帮 Agent 节省 95% 的 API 账单?
- 一个技巧，让你省下60%的token

例如 模型缓存：

- Deepseek-renox

例如：架构优化

- 贵的好的模型做推理规划、便宜的国产模型做推理和执行

目标

- 输出 Claude、GPT 等大模型的优化措施、方案和管控工具
- 输出任务自动路由方案和模型自动选择方案
- 精细化管控方案

方案

必须建立的管理动作：账单透明、部门分摊、管理层审批（需要给管理层提供数据支撑，哪些模型能做什么，不能做什么，他的员工的任务是否真的必须 Opus） 堵。…… 。

方案一：

研究 Claude Code/Oh-my-pi 等 Agent 为啥消耗 Token 这么快，探索可控、可预支的 Token消耗和节约经验。如果能做到成本可控，可以局部放开，使用 Opus 做推理规划、执行干活用国产。需要研究混合模型使用方式，例如 OpenCode 的方式、KuAgent 的方式。

工具 Claude-tap，个人监控工具。

方案二：

快速引入国产模型（Kimi-2.6、MiniMax3、Qwen3.7-Max-Plus 等），并研究各种使用场景和内部任务的 benchmark 效果。给出任务-模型适配度。

方案三：Langfuse 可观测性体系建设

> 🔭 **核心问题**：当前 Token 消费是"黑箱"，无法知道谁在用什么客户端、消耗了多少、哪些调用是浪费的。

序号	需求	具体要求
3.1	**客户端识别与分类**：在 Langfuse 中追踪每个请求来自哪个客户端（Claude Code、Cursor、自研 Agent、API 直调等）	通过 metadata 标记 client_type / client_version / user_id，实现按客户端维度的消费归因
3.2	**高消费客户端挖掘**：自动识别 Token 消耗 Top N 的客户端/用户/任务类型	Langfuse Dashboard 按客户端聚合 Token 消耗 + 异常检测（单日突增预警）+ 消费趋势对比
3.3	**优化机会挖掘**：从 Langfuse trace 数据中，自动发现可优化的调用模式	识别：重复调用、过长上下文、低效 Prompt、可缓存未缓存、可降级未降级等模式

方案四：上下文工程（Context Engineering）

🧠 **核心问题**：上下文不是越多越好。Chroma 研究表明，模型性能随输入长度增加而**非均匀退化**（Context Rot），更长的上下文 = 更差的质量 + 更高的成本。

序号	需求	具体要求
4.1	**上下文组装策略研究**：研究如何在有限窗口内最大化"有效信息密度"	设计上下文优先级框架：系统指令 > 任务描述 > 相关文件 > 历史对话 > 辅助信息。每层有预算上限和淘汰策略
4.2	**无关信息剔除策略**：在不影响 KV Cache 命中率的前提下，剔除无用上下文	研究"安全裁剪区"：哪些位置的内容可以裁剪而不影响 cache prefix；设计基于语义相似度的裁剪算法；量化裁剪前后的质量与成本变化
4.3	**上下文压缩技术**：研究并实现上下文压缩方案（摘要替代、工具输出精简、对话历史压缩）	对比 3+ 种压缩策略的效果（Token 减少率 vs 质量影响），给出推荐方案和适用场景

方案五：AI 网关（CCH）KV Cache 优化

⚡ **核心问题**：KV Cache（Prompt Caching）是大模型最重要的降本机制（缓存命中时成本降低 90%），但实际命中率往往不高。网关层有巨大的优化空间。

序号	需求	具体要求
5.1	**网关层 Prefix 稳定性优化**：在 CCH 网关层确保每次请求的前缀（System Prompt + Tool Definitions）保持稳定一致	网关自动检测并排序 message 顺序，将稳定部分前置；监控前缀变化率（前缀变动 = cache miss）；设计"前缀锁定"机制
5.2	**客户端侧缓存友好设计**：指导客户端（Claude Code、Cursor 等）如何构造 cache-friendly 的请求	输出《客户端 KV Cache 最佳实践》：System Prompt 固定化、Tool 定义排序稳定、上下文追加而非替换、避免随机打乱消息顺序
5.3	**KV Cache 命中率监控**：在网关层实时监控 cache_creation_tokens vs cache_read_tokens 比例	按客户端/用户/模型维度的缓存命中率看板 + 低命中率告警 + 自动诊断 miss 原因（前缀变动/消息重排/超时失效）

5.4	**不影响用户性能的剔除策略**：研究在保持 KV Cache 命中的同时，安全移除无用上下文	关键约束：只能修改"缓存前缀之后"的内容；前缀部分只做"追加"不做"删除/修改"；设计 safe-trim 算法：识别哪些内容在 prefix 之外、可以安全裁剪

方案六：上下文腐烂（Context Rot）与模型蠢笨区治理

> 💡 **核心问题**：Chroma 2025 年研究（评估 18 个 LLM）证实——模型性能随输入长度增加而**非均匀退化**。即使任务复杂度不变，仅增加无关内容就会导致性能下降。这在编程场景中尤为严重，表现为模型进入"蠢笨区"：忘记指令、重复生成、逻辑断裂。

序号	需求	具体要求
6.1	**Context Rot 量化研究**：在企业编程场景中复现 Chroma 实验，量化我们常用模型的退化曲线	在 5+ 编程任务上（代码生成、Bug 修复、重构、代码审查、文档生成），按输入长度梯度测试 Claude Opus/Sonnet/Haiku 的性能退化点（从哪个长度开始退化、退化速度如何）
6.2	**蠢笨区检测与预警**：定义"蠢笨区"的量化指标，实现自动检测	指标设计：当输入 Token 超过某阈值后，模型输出质量（正确率、一致性、指令遵循度）显著下降的区间。实现实时监控 + 进入蠢笨区时自动预警（建议开启新会话或裁剪上下文）
6.3	**编程场景治理方案**：针对编程场景的蠢笨区，设计系统级治理策略	策略包括：(1) 会话生命周期管理（自动建议在退化点前总结并开新会话）(2) 上下文窗口预算管理（为每个任务类型设置最大上下文预算）(3) 渐进式上下文策略（初期宽松，接近退化点时激进裁剪）(4) 多会话协作模式（复杂任务拆分为多个短会话）
6.4	**干扰物（Distractor）影响研究**：Chroma 研究证实干扰物会非均匀地加剧性能退化	研究编程场景中的"干扰物"是什么（无关文件内容、过时的对话历史、冗余的工具输出等），量化不同干扰物对编程任务的影响程度，设计干扰物识别与过滤机制

(注：内容由 AI 生成，请谨慎参考)